

Testing & Quality Gates Question Index - All 50 Questions

Q#	Question
Q1	Testing types - unit, integration, E2E, smoke, regression
Q2	Testing pyramid - cost trade-offs, why more unit tests
Q3	TDD - red-green-refactor, benefits, when to use
Q4	BDD - Given/When/Then, Gherkin, Cucumber
Q5	Code coverage - line, branch, function, what % is enough
Q6	SonarQube basics - quality gates, rules, profiles, debt
Q7	SonarQube metrics - bugs, vulnerabilities, smells
Q8	JUnit basics - annotations, assertions, lifecycle
Q9	PyTest basics - fixtures, parametrize, markers
Q10	Mocha/Jest - describe/it, async, mocking
Q11	Selenium - WebDriver, locators, waits, page objects
Q12	Postman - collections, environments, assertions
Q13	Newman - Postman in CI/CD, reports
Q14	Test environments - data management, parity, isolation
Q15	Shift-left testing - earlier testing, developer ownership
Q16	SonarQube quality gates - pass/fail, blocking CI
Q17	SonarQube CI integration - PR decoration, branch analysis
Q18	SonarQube custom rules - writing rules, exclusions
Q19	Static analysis - ESLint, Pylint, Checkstyle, PMD
Q20	JUnit advanced - parameterized, Mockito, extensions
Q21	PyTest advanced - fixture scope, plugins, parallel
Q22	Cypress - auto-waiting, time travel, intercepts
Q23	Playwright - cross-browser, codegen, trace viewer
Q24	Cypress vs Selenium vs Playwright comparison

Q25	Postman advanced - scripts, chaining, data-driven
Q26	API contract testing - Pact, consumer-driven contracts
Q27	Performance testing - JMeter, k6, Locust, types
Q28	Chaos testing - Chaos Monkey, Litmus, Game Days
Q29	Mocking - WireMock, MockServer, when to mock
Q30	Testcontainers - real services for integration tests
Q31	Test reporting - Allure, HTML, CI dashboards
Q32	Flaky tests - causes, detection, quarantine
Q33	Test data management - factories, fixtures, masking
Q34	Security testing in pipeline - SAST, DAST integration
Q35	Infrastructure testing - Molecule, InSpec, Goss, Terratest
Q36	Testing strategy - what to test at which level
Q37	Quality gates design - balancing speed and quality
Q38	Test automation ROI - what to automate, cost-benefit
Q39	Testing microservices - contracts, virtualization
Q40	Testing in production - canary, feature flags, A/B
Q41	Compliance testing - regulatory, audit evidence
Q42	Testing culture - quality ownership, shared responsibility
Q43	SonarQube gate blocking release
Q44	Flaky E2E test failing randomly
Q45	Coverage dropped 80% to 45%
Q46	Selenium tests taking 2 hours
Q47	API contract broken after deployment
Q48	Performance test shows 10x latency
Q49	Test environment data corrupted
Q50	Design testing strategy for 30 microservices

SECTION 1 - Junior Core Testing (Q1–Q15)

Q1 Testing types - unit, integration, E2E, smoke, regression

The Simple Answer

Testing types define WHAT you test and HOW deep you go. Each type catches different bugs at different costs. Knowing which type to use where is a fundamental DevOps skill.

The Analogy - Building a Car

Unit tests check individual parts - does the engine start? Does the brake pad grip? Integration tests check parts working together - does the engine connect properly to the transmission? Does the brake system respond to the pedal? E2E tests drive the entire car on a road - start the car, drive to a destination, park. Smoke tests check after assembly - does it turn on at all? Regression tests verify that fixing the brakes did not break the steering.

Each Type in Detail

Unit tests - Test a single function or class in complete isolation. All dependencies are mocked. Execution: milliseconds. You can run thousands per minute. They catch logic bugs - a calculation is wrong, an edge case is not handled, a null check is missing. Example: test that `calculateDiscount(100, 0.2)` returns 80, that `calculateDiscount(100, 0)` returns 100, and that `calculateDiscount(100, -1)` throws an error.

Integration tests - Test multiple components working together with real dependencies. A service calls a real database, a real message queue, or a real API endpoint. Execution: seconds. They catch interface bugs - the SQL query has a syntax error that only appears with a real database, the message serialization format does not match what the consumer expects. Example: test that the order service stores an order in PostgreSQL and publishes an event to Kafka that the payment service can consume.

End-to-End (E2E) tests - Test the entire application flow from the user's perspective. A real browser clicks buttons, fills forms, navigates pages. Execution: minutes per test. Most realistic but most fragile, slowest, and most expensive to maintain. Example: a user logs in, searches for a product, adds it to cart, enters shipping info, pays, and receives a confirmation email.

Smoke tests - Quick sanity checks after deployment. Not comprehensive - just "is the system alive?" Does the health endpoint return 200? Can it connect to the database? Can it reach dependent services? Run after every deployment. If smoke tests fail, rollback immediately. Execution: seconds.

Regression tests - Verify that new changes did not break existing functionality. Typically the full test suite run before release. "Everything that worked before still works." Not a separate type - it is running your existing unit, integration, and E2E tests to catch regressions.

Acceptance tests - Verify business requirements are met. Written in collaboration with product owners. "As a user, I can reset my password via email." Often use BDD format (Given/When/Then).

Q2 Testing pyramid - cost trade-offs

The Simple Answer

The testing pyramid recommends: many unit tests at the base, fewer integration tests in the middle, and fewest E2E tests at the top. Each level up trades speed and stability for realism.

The Analogy - Quality Control in a Factory

Unit tests are like checking each component on the assembly line - fast, cheap, catches most defects. Integration tests are like testing assembled subsystems - slower but catches connection issues. E2E tests are like driving the finished car off the lot - the ultimate test but expensive and time-consuming. A factory that only does test drives (E2E) and skips component checks (unit) will find defects late and expensively.

Why This Shape Matters

Unit tests (70% of your tests) - Fast (milliseconds), cheap to write, stable (no flakiness from external dependencies), and highly specific (when they fail, you know exactly which function broke). Run 10,000 unit tests in 30 seconds. Catch 70% of bugs.

Integration tests (20%) - Slower (seconds per test), need infrastructure (databases, queues, APIs), broader coverage. Run hundreds in minutes. Catch bugs that unit tests miss: SQL syntax errors, serialization mismatches, transaction behavior.

E2E tests (10%) - Slowest (minutes per test), fragile (break on UI changes, timing issues, network problems), expensive to maintain, and flaky. Run dozens in tens of minutes. Catch the remaining bugs: CSS that hides a button, a redirect that loops, a workflow that fails across multiple services.

The Anti-Pattern: Ice Cream Cone

Many E2E tests, few unit tests (inverted pyramid). Results in: slow CI pipelines (2+ hours), flaky builds (E2E tests fail randomly 10% of the time), hard to debug ("the checkout test failed" - but which of the 12 services involved broke?), and expensive maintenance (UI changes break 50 E2E tests).

****The interview question:**** "Your E2E suite takes 2 hours and fails 20% of the time. What do you do?" Answer: Identify what those E2E tests are testing. Move detailed logic tests to unit level. Keep only critical user flows as E2E. Parallelize what remains. Target: 70/20/10 split. See scenario Q46.

Q3 TDD - red-green-refactor

The Simple Answer

Test-Driven Development writes tests BEFORE writing code. The cycle repeats for every small piece of functionality.

The Cycle

Red - Write a test for behavior that does not exist yet. Run it - it fails (red). This confirms the test is actually testing something. A test that passes before you write the code is testing nothing.

Green - Write the MINIMUM code to make the test pass. Not elegant, not optimized - just enough to pass. Resist the urge to write more.

Refactor - Clean up the code (remove duplication, improve naming, extract functions) while keeping all tests green. The tests give you confidence that refactoring did not break anything.

The Analogy

TDD is like building a house by first drawing the inspection checklist, then building just enough to pass each inspection, then cleaning up. Without TDD, you build the house first and hope it passes inspection - finding out problems after the walls are up is expensive.

Benefits in Practice

Forces clear requirements before coding - "what should this function return for these inputs?" Produces testable code by design - code written to pass tests is naturally modular and decoupled. Creates a comprehensive test suite as a side effect - every feature has tests from day one. Prevents over-engineering - you only write code that makes a test pass. Increases confidence in refactoring - tests catch regressions immediately.

When TDD Excels

Business logic with clear inputs and outputs. Algorithm development. Bug fixes (write a test that reproduces the bug, then fix it - guaranteed to not regress). Library and API development. Any code where the requirements are clear before coding.

When TDD Is Less Practical

UI development (visual output hard to test-first). Exploratory prototyping (requirements unclear - you are discovering them). Infrastructure code (Terraform, Docker - other testing approaches like Terratest are better). Integration points where the interface is still being designed.

Q4 BDD - Given/When/Then

The Simple Answer

Behavior-Driven Development describes tests in business language that non-developers can read, write, and review. It bridges the gap between business requirements and automated tests.

Gherkin Format

Feature: User Login. Scenario: Successful login with valid credentials. Given the user is on the login page. And the user has a valid account. When they enter their username and password. And click the login button. Then they should see the dashboard. And a welcome message should display their name.

The Analogy

TDD speaks the language of developers (functions, assertions, mocks). BDD speaks the language of business (given this situation, when this action happens, then this result should occur). BDD is TDD translated for stakeholders - the scenarios become executable specifications that both the product owner and the computer understand.

Tools

Cucumber - The standard BDD framework. Supports Java, JavaScript, Ruby, Python. Maps Gherkin steps to step definition code. Each "Given/When/Then" line maps to a code function. Behave - Python BDD framework. Same Gherkin syntax. SpecFlow - .NET BDD framework.

Real-World Value

Product owners write and review acceptance criteria in Gherkin before development starts. Developers implement the step definitions. QA reviews the scenarios for completeness. The Gherkin files serve as living documentation - always up to date because they ARE the tests. When a scenario fails, everyone understands what broke in business terms.

Q5 Code coverage - what percentage is enough

The Simple Answer

Code coverage measures what percentage of your code is executed during tests. It answers: "how much of my code is tested?" But NOT "how well is it tested?"

Coverage Types

Line coverage - Percentage of lines executed. Simplest but misleading - a line can execute without testing all code paths through it. Branch coverage - Percentage of if/else branches taken. More meaningful - ensures both sides of every conditional are tested. Function coverage - Percentage of functions called. Least detailed but gives a high-level view of untested areas.

The Analogy

Coverage is like a security camera that shows which rooms were visited. 80% coverage means cameras captured activity in 80% of rooms. But it does not tell you whether anything was CHECKED in those rooms - someone might have walked through the room without examining anything. Coverage tells you where tests went, not what they verified.

What Percentage Is Enough

80% is the widely accepted target. Below 60% indicates significant untested code - real risk of undetected bugs. 80-90% covers the important logic with reasonable effort. Above 90% often means diminishing returns - testing trivial code (getters, setters, constructors) to hit the number without adding value. 100% is rarely worth pursuing and does not guarantee bug-free code - you can have 100% line coverage and still miss edge cases if assertions are weak.

****The golden rule:**** Coverage tells you what is NOT tested - use it to find gaps. Do not use it as a quality metric. A project with 60% coverage and excellent assertion quality is more reliable than one with 95% coverage and no meaningful assertions.

Q6 SonarQube basics - quality gates, rules, profiles

The Simple Answer

SonarQube is a continuous code quality and security analysis platform. It scans source code and reports: bugs (code that will likely break), vulnerabilities (security weaknesses), code smells (maintainability issues), duplication (copy-pasted code), and coverage (test coverage percentage). It integrates into CI/CD pipelines to enforce quality standards.

The Analogy

SonarQube is like a building inspector who examines your code after every change. The inspector checks: structural integrity (bugs), fire safety (vulnerabilities), maintenance accessibility (code smells), and building code compliance (quality gate). If the inspection fails, you cannot move in (deploy).

Key Concepts

Quality Gate - A pass/fail threshold for code quality. The default "Sonar Way" gate requires on new code: 0 bugs (A rating), 0 vulnerabilities (A rating), less than 3% duplication, greater than 80% coverage, and 0 unreviewed security hotspots. If any condition fails, the gate fails, and the CI pipeline fails - code cannot be merged.

Rules - Individual checks. SonarQube has thousands per language: "Unused variables should be removed," "SQL queries should use parameterized statements," "Passwords should not be hardcoded." Each rule has a severity: Blocker, Critical, Major, Minor, Info.

Quality Profiles - Collections of rules applied to a project. Choose which rules matter for your team. Start with SonarQube's defaults (Sonar Way profile) and customize. Different projects can use different profiles - a security-critical payment service uses stricter rules than an internal admin tool.

Technical Debt - The estimated time to fix all code smells. SonarQube quantifies it: "42 hours of technical debt in this project." Helps prioritize cleanup work and track improvement over time.

Q7 SonarQube metrics - bugs, vulnerabilities, smells

The Simple Answer

SonarQube categorizes issues into distinct types, each requiring different urgency levels.

Bugs - Code that will likely behave incorrectly at runtime. Null pointer dereferences, resource leaks, incorrect equality comparisons, infinite loops under certain conditions. Rated by severity. Fix immediately - these are real defects that will cause production incidents.

Vulnerabilities - Security weaknesses that an attacker could exploit. SQL injection, cross-site scripting (XSS), hardcoded credentials, weak cryptography, insecure deserialization. Fix before deployment - these are exploitable attack vectors.

Security Hotspots - Code that requires manual security review. Not necessarily vulnerable but security-sensitive: crypto usage, file permissions, regex patterns (ReDoS risk). A human reviews each hotspot and marks it as safe or vulnerable.

Code Smells - Maintainability issues that are not bugs or security risks today but will slow development over time. Long methods (>50 lines), deep nesting (>3 levels), duplicated logic, unused imports, overly complex functions (high cyclomatic complexity). Fix during regular maintenance sprints.

Duplication - Percentage of code that is copy-pasted. High duplication means fixing a bug in one place but missing the identical code elsewhere. Target: below 3% on new code.

Coverage - Percentage of code covered by unit tests. Reported by test frameworks (JaCoCo for Java, coverage.py for Python, Istanbul/c8 for JavaScript) and consumed by SonarQube for quality gate evaluation.

Ratings

SonarQube rates each category A through E. A is best (no issues or minimal). E is worst. Quality gates typically require A rating on new code for reliability (bugs), security (vulnerabilities), and security review (hotspots).

Q8 JUnit basics - annotations, assertions

The Simple Answer

JUnit is the standard testing framework for Java and JVM languages. Tests are methods annotated with `@Test` that verify behavior through assertions.

Key Annotations (JUnit 5)

`@Test` - Marks a method as a test. `@BeforeEach` - Runs before EACH test method (setup - create fresh objects, reset state). `@AfterEach` - Runs after EACH test (cleanup - close connections, delete temp files). `@BeforeAll` - Runs once before ALL tests in the class (static method - expensive one-time setup like starting a container). `@AfterAll` - Runs once after ALL tests. `@DisplayName("Human readable description")` - Gives the test a descriptive name in reports. `@Disabled("Reason")` - Skips the test with a documented reason.

Assertions

`assertEquals(expected, actual)` - Values are equal. `assertTrue(condition)` - Condition is true.
`assertThrows(Exception.class, () -> riskyMethod())` - Code throws the expected exception. `assertNotNull(object)` - Object is not null. `assertTimeout(Duration.ofSeconds(5), () -> slowMethod())` - Code completes within time limit.
`assertAll()` - Run multiple assertions, report all failures (not just the first).

The Analogy

JUnit is like a systematic quality inspector with a checklist. `@BeforeEach` prepares the inspection station for each item. `@Test` is the inspection itself. Assertions are the pass/fail criteria. `@AfterEach` cleans the station for the next item. The inspector does not stop at the first defect - `assertAll` reports all defects in one pass.

Q9 PyTest basics - fixtures, parametrize, markers

The Simple Answer

PyTest is Python's most popular testing framework. Minimal boilerplate - test functions just start with `test_`. Powerful fixture system for setup and teardown. Rich plugin ecosystem.

Fixtures - The Power Feature

A fixture provides reusable setup code with automatic cleanup. `@pytest.fixture` `def database(): conn = create_connection(); yield conn; conn.close()`. Tests that need a database receive it as a parameter: `def test_query(database): result = database.execute("SELECT 1")`. The `yield` separates setup (before `yield`) from teardown (after `yield`). Teardown runs even if the test fails - guaranteed cleanup.

Parametrize - One Test, Many Inputs

`@pytest.mark.parametrize("input,expected", [(2,4), (3,9), (4,16), (0,0), (-2,4)])`. One test function, five test cases. Each runs independently - if `input=3` fails, the others still run. Eliminates writing five nearly identical test functions.

Markers - Labeling Tests

@pytest.mark.slow, @pytest.mark.integration, @pytest.mark.smoke. Run subsets: pytest -m "not slow" skips slow tests during development. pytest -m smoke runs only smoke tests after deployment. pytest -m integration runs only integration tests in CI.

conftest.py

Shared fixtures defined here are available to all tests in the directory and subdirectories without import statements. Place common fixtures (database connections, API clients, test data) in conftest.py. Pytest discovers and injects them automatically.

Q10 Mocha/Jest - JavaScript testing

The Simple Answer

Jest - Facebook's testing framework. Zero configuration for React projects. Built-in mocking, assertions, and coverage. Runs tests in parallel (each file in a separate worker). Snapshot testing for UI components. The default choice for modern JavaScript.

Mocha - Flexible, mature testing framework. Requires separate assertion library (Chai) and mocking library (Sinon). More configuration but more control over the test environment. Popular for Node.js backend services.

Test Structure

describe("Calculator", () => { describe("add", () => { it("should add two positive numbers", () => { expect(add(2, 3)).toBe(5); }); it("should handle negative numbers", () => { expect(add(-1, 1)).toBe(0); }); }); });. describe groups related tests. it defines individual test cases. expect/toBe are assertions.

Async Testing

Modern JavaScript is heavily asynchronous. Jest handles it naturally: test("fetches user data", async () => { const user = await fetchUser(1); expect(user.name).toBeDefined(); expect(user.email).toContain("@"); });. For API calls, database queries, and any promise-based code.

Mocking in Jest

jest.mock("./database") replaces the real database module with a mock. jest.fn() creates a mock function. mockReturnValue(42) defines what the mock returns. Mocking isolates tests from external dependencies - no real database needed for unit tests.

Q11 Selenium - WebDriver, locators, waits

The Simple Answer

Selenium automates web browsers for E2E testing. WebDriver controls a real browser (Chrome, Firefox, Safari, Edge) programmatically - clicking buttons, filling forms, navigating pages, and verifying content.

The Analogy

Selenium is like hiring a very fast, very literal human to test your website. You give exact instructions: "Go to this URL. Find the button with ID 'login'. Click it. Wait for the dashboard to load. Check that the text 'Welcome' is visible." The human follows instructions exactly but cannot improvise - if the button ID changes to 'login-btn', the instruction fails.

Locators - Finding Elements

By.ID - `driver.find_element(By.ID, "login-btn")`. Fastest and most stable - IDs should be unique. By.CSS_SELECTOR - `driver.find_element(By.CSS_SELECTOR, ".submit-button")`. Flexible, familiar syntax. By.XPATH - `driver.find_element(By.XPATH, "//button[@type='submit']")`. Most powerful but most fragile - complex XPaths break easily. Preference order: ID > CSS > XPath. Use the simplest locator that uniquely identifies the element.

Waits - The #1 Selenium Problem

Web pages load asynchronously. An element might not exist when Selenium tries to find it. Implicit wait - `WebDriverWait` globally for N seconds for any element. Applies to all find operations. Simple but imprecise. Explicit wait - Wait for a specific condition: `WebDriverWait(driver, 10).until(EC.element_to_be_clickable((By.ID, "btn")))`. Precise. Preferred. Never use `time.sleep()` - It always waits the full duration even if the element appears instantly. Wastes time. Causes flaky tests when the page is slower than expected.

Page Object Model (POM)

Encapsulate page interactions in classes. `LoginPage` has methods: `enter_username("alice")`, `enter_password("secret")`, `click_submit()`, `get_error_message()`. Tests call page methods: `login_page.enter_username("alice"); login_page.click_submit()`. When the UI changes, update the page object - not every test that uses it. Dramatically reduces maintenance.

Q12 Postman - collections, environments

The Simple Answer

Postman is the most popular API testing tool. Create HTTP requests, organize them into collections, run them against different environments, and verify responses with automated assertions.

The Analogy

Postman is like a customer testing a restaurant. You place an order (send a request), specify the restaurant location (environment - dev, staging, prod), and check if you got the right dish (assertions on the response). Collections are your full order list for the evening - appetizer, main, dessert - tested in sequence.

Collections

Groups of requests organized by API or workflow. "User API" collection: Create User (POST), Get User (GET), Update User (PUT), Delete User (DELETE). Collections can be shared with the team, version-controlled, and run automatically.

Environments

Variable sets per deployment environment. dev: `base_url = http://localhost:3000`, `api_key = dev-key-123`. staging: `base_url = https://staging.api.com`, `api_key = staging-key-456`. Switch environments - same requests, different servers. No manual URL editing.

Test Assertions

```
pm.test("Status code is 200", () => { pm.response.to.have.status(200); }); pm.test("Response time under 500ms", () => { pm.expect(pm.response.responseTime).to.be.below(500); }); pm.test("User name is correct", () => { const json = pm.response.json(); pm.expect(json.name).to.eql("Alice"); pm.expect(json.email).to.include("@"); });
```

Q13 Newman - Postman in CI/CD

The Simple Answer

Newman is Postman's command-line collection runner. It takes exported Postman collections and runs them in CI/CD pipelines - no GUI needed. This transforms Postman from a manual testing tool into an automated API testing framework.

Usage

`newman run collection.json -e staging-env.json --reporters cli,html,junit`. Runs the collection against the staging environment. Produces CLI output (terminal), HTML report (for humans), and JUnit XML (for CI tools). Exit code 0 = all tests passed. Non-zero = tests failed - CI pipeline fails.

CI/CD Integration

Export the collection and environment from Postman. Add them to the repository under `tests/api/`. CI pipeline step: `npm install -g newman && newman run tests/api/collection.json -e tests/api/staging.json --reporter-junit-export results.xml`. Pipeline fails if any assertion fails. JUnit report is consumed by Jenkins, GitLab, or GitHub Actions for test result display.

****The workflow:**** Developers create and test API requests in Postman GUI (fast, visual feedback). Newman runs the same tests in CI (automated, every PR). API tests are part of the quality gate - broken APIs block deployment.

Q14 Test environments - data, parity, isolation

The Simple Answer

Test environments must be: isolated (tests in dev do not affect staging), representative (mirror production as closely as possible), and reproducible (rebuild from scratch in minutes).

The Analogy

A test environment is like a rehearsal stage for a play. It should look exactly like the real theater (production parity), each rehearsal should start with a clean stage (isolation), and you should be able to set up the stage quickly for multiple rehearsals per day (reproducibility). Rehearsing on a different stage with different props gives false confidence - everything "works" in rehearsal but fails on opening night.

Test Data Strategies

Seed data - Known data loaded before tests from version-controlled fixtures (SQL files, JSON). Predictable and repeatable. Reset between test runs. Data factories - Generate test data programmatically. Factory Boy (Python), Faker (any language) create realistic but fake data: names, addresses, emails, phone numbers. Each test gets unique data - no conflicts. Data masking - Copy production data with PII replaced by realistic fakes. Preserves data distribution, relationships, and edge cases. Tools: Tonic, Delphix, `pg_anonymize`. Used when tests need production-

like data volumes. Database per test (Testcontainers) - Each test gets a fresh database in a Docker container. Complete isolation. No cleanup needed. Slightly slower but eliminates shared state bugs entirely. See Q30.

Q15 Shift-left testing - earlier testing

The Simple Answer

Shift-left means testing EARLIER in the software development lifecycle. Instead of testing after development is "done," testing happens during and before development. The earlier a bug is found, the cheaper it is to fix.

The Analogy

Traditional testing is like proofreading a book after it is printed - finding a typo requires reprinting thousands of copies. Shift-left testing is like spell-checking as you type - errors are caught and fixed in seconds. The cost difference: fixing a bug during coding takes 10 minutes. Fixing the same bug in production takes days (investigation, hotfix, deployment, customer communication).

Shift-Left in Practice

In the IDE - Linting catches errors as you type. ESLint, Pylint, IntelliJ inspections highlight issues before you even save the file. Pre-commit hooks - Run unit tests, linters, and secret scanners before the commit is accepted. Bad code never enters the repository. PR pipeline - Unit tests + integration tests + SAST + SCA run on every pull request. Results posted directly on the PR as comments. Merge is blocked until all checks pass. Developer-written tests - Developers own their tests. No "throw it over the wall to QA." The person who wrote the code understands it best and writes the most effective tests. TDD - The ultimate shift-left: tests are written BEFORE the code. See Q3.

SECTION 2 - Mid-Level Production Patterns (Q16–Q35)

Q16 SonarQube quality gates - blocking CI

The Simple Answer

Quality gates are the enforcement mechanism that prevents bad code from reaching production. The gate evaluates metrics on new code and returns pass or fail. When integrated with CI/CD, a failed gate blocks the merge - the PR cannot be completed.

Default Sonar Way Gate Conditions

On new code: 0 new bugs (Reliability rating A), 0 new vulnerabilities (Security rating A), less than 3% duplication, greater than 80% test coverage, 0 unreviewed security hotspots (Security Review rating A). These conditions are strict but achievable - they apply only to NEW code, not the entire codebase.

Focus on New Code - The Key Insight

You cannot fix a 200,000-line legacy codebase overnight. But you CAN ensure every new change meets the standard. SonarQube's "new code" concept (changes since the last version or a specific date) enables this. Over time, as new code replaces old code, the overall quality improves organically - without a dedicated "tech debt cleanup" project.

CI Pipeline Integration

The pipeline runs: sonar-scanner (analyzes code) → SonarQube evaluates the quality gate → pipeline checks the gate status → pass: continue → fail: pipeline fails. In Jenkins: `waitForQualityGate()`. In GitHub Actions: check SonarQube API. In GitLab CI: use the SonarQube quality gate webhook or API.

Q17 SonarQube CI integration - PR decoration

The Simple Answer

PR decoration posts SonarQube analysis results directly on pull requests as comments. Developers see: new bugs introduced in this PR, new vulnerabilities, coverage on changed lines, duplication, and the overall quality gate result - all without leaving GitHub or GitLab.

How It Works

SonarQube performs branch analysis. Short-lived branches (feature branches) are compared against the target branch (main). The analysis shows only the DELTA - issues introduced in THIS PR, not issues in the existing codebase. This is critical: developers only see issues they can fix, not legacy problems they did not create.

Branch Analysis Types

Short-lived branches - Feature branches. Compared against the target (main). Only new issues shown. Quality gate evaluates new code only. Long-lived branches - Release branches, main branch. Full analysis with their own quality baseline. Track overall project health over time.

****The developer experience:**** Push code → CI runs SonarQube → PR shows: "3 new bugs, 1 vulnerability, coverage 72% (below 80% threshold). Quality gate: FAILED." The developer fixes the issues, pushes again → "0 bugs, 0 vulnerabilities, coverage 85%. Quality gate: PASSED." All visible in the PR without switching contexts.

Q18 SonarQube custom rules - writing rules, exclusions

The Simple Answer

Beyond SonarQube's 5000+ built-in rules, organizations add custom rules for: internal coding standards ("all REST controllers must have @ApiVersion annotation"), domain-specific patterns ("database connection strings must use the connection pool"), or regulatory requirements ("logging must not contain PII fields").

Writing Custom Rules

Java-based: extend SonarQube's API using the Custom Rules plugin. Define AST (Abstract Syntax Tree) patterns to match. For example: detect any method that catches Exception (too broad) instead of specific exceptions. Publish as a SonarQube plugin.

Importing External Results

SonarQube imports findings from external tools: ESLint results (via sonar.eslint.reportPaths), Checkstyle XML, PMD reports, and custom tool output in the Generic Issue Import Format. This lets you keep your existing linters and aggregate results in SonarQube's dashboard.

Exclusions

Exclude files from analysis: generated code (protobuf, API clients), vendor/third-party code, test fixtures, migration scripts. Configure in sonar-project.properties: sonar.exclusions=/generated/, /vendor/. Exclusions prevent false positives and keep metrics accurate.

Q19 Static analysis - ESLint, Pylint, Checkstyle, PMD

The Simple Answer

Static analysis examines code without executing it. Each language has specialized linters that catch different issues.

ESLint - JavaScript/TypeScript. Highly configurable rules. Auto-fix for many issues (eslint --fix). Plugins for React, Vue, Node.js. The universal standard for JS projects. Pylint / flake8 / Ruff - Python. Pylint is comprehensive but slow. flake8 is simpler and faster. Ruff is the modern choice - written in Rust, 10-100x faster than Pylint, covers both linting and formatting, and is rapidly replacing both Pylint and Black. Checkstyle - Java code style enforcement. Indentation, naming conventions, import ordering. PMD - Java bug detection. Unused variables, empty catch blocks, complex code. golangci-lint - Go. Aggregates 50+ linters into a single tool. The standard for Go projects. ShellCheck - Bash scripts. Catches: unquoted variables, missing error handling, incorrect test syntax, portability issues. Essential for any team writing shell scripts.

Integration Pattern

Run linters in two places: pre-commit hooks (immediate feedback - fix before committing) AND in CI (enforcement - cannot merge if linter fails). Auto-fix where possible: eslint --fix, ruff --fix. Treat warnings as errors in CI - if it is not worth fixing, disable the rule; do not ignore warnings.

Q20 JUnit advanced - parameterized, Mockito

Parameterized Tests

@ParameterizedTest runs one test method with multiple inputs. @ValueSource(ints = {1, 2, 3, 4, 5}) provides simple values. @CsvSource({"1,1,2", "2,3,5", "10,20,30"}) provides multi-argument cases.

@MethodSource("provideTestData") references a method that returns a Stream of Arguments - for complex test data. One test method, many test cases. Each case runs independently and reports separately.

Mockito - Isolating Dependencies

@Mock - Creates a mock implementation of an interface or class. No real behavior. @InjectMocks - Creates the class under test and injects mocks into its constructor/fields. when(mock.findById(1L)).thenReturn(Optional.of(testUser)) - Defines mock behavior: when findById is called with 1, return testUser.
verify(emailService).sendWelcome(testUser) - Asserts that the mock method was called with the expected argument. ArgumentCaptor<Order> captor - Captures the argument passed to a mock method for detailed assertion: verify(orderRepo).save(captor.capture()); assertEquals("PENDING", captor.getValue().getStatus());

****The mock philosophy:**** Mock what you do not own (external APIs, databases in unit tests). Do not mock what you ARE testing. A test that mocks everything tests nothing - it only tests that the mocks are wired correctly.

Q21 PyTest advanced - fixture scope, parallel

Fixture Scopes

function (default) - Created fresh for EACH test. Best isolation. Use for: mutable test data, database transactions.
class - Shared across all tests in a class. Use for: read-only test data.
module - Shared across all tests in a file. Use for: expensive read-only setup (loading a large dataset).
session - Created ONCE for the entire test run. Use for: starting a Docker container, creating a database schema, connecting to an external service.

Parallel Execution with pytest-xdist

pytest -n auto distributes tests across CPU cores. pytest -n 4 uses 4 workers. A 20-minute test suite on 4 cores finishes in roughly 5 minutes. Requirement: tests must be independent - no shared mutable state between tests running in parallel.

Essential Plugins

pytest-cov - Coverage reporting: pytest --cov=myapp --cov-report=html. pytest-mock - Mockito-style mocking for Python: mocker.patch("myapp.db.connect"). pytest-asyncio - Test async/await code. pytest-timeout - Kill tests exceeding a time limit: @pytest.mark.timeout(30). pytest-randomly - Randomize test order to detect order-dependent tests - if a test only passes when run after another specific test, random ordering exposes it.

Q22 Cypress - modern E2E testing

The Simple Answer

Cypress is a modern E2E testing framework that runs INSIDE the browser alongside your application. Unlike Selenium (which controls the browser from outside via WebDriver), Cypress has direct access to the DOM, network requests, and application state.

The Analogy

Selenium is like controlling a robot arm to press buttons on a vending machine from outside. Cypress is like being INSIDE the vending machine - you see every gear turn, every signal fire, and can intercept any operation. This inside access makes Cypress faster, more reliable, and more debuggable.

Key Features

Auto-waiting - Cypress automatically waits for: elements to exist in the DOM, elements to become visible, elements to stop animating, XHR/fetch requests to complete. No explicit waits needed. No flaky timing issues.

Time travel debugging - Cypress takes DOM snapshots at every step. In the test runner, hover over any step to see exactly what the page looked like. Click to inspect the DOM at that point. This is the most powerful debugging feature in any E2E framework.

Network interception - `cy.intercept('GET', '/api/users', { fixture: 'users.json' })` mocks API responses. Test the UI with controlled data - no real backend needed. Test error handling: `cy.intercept('/api/order', { statusCode: 500 })`. Test loading states: `cy.intercept('/api/data', { delay: 3000, body: {...} })`.

Limitations

Chrome-family and Firefox only (no Safari in the open-source version - Cypress Cloud adds WebKit). Same-origin limitation (cannot test across different domains in one test). No native mobile testing.

Q23 Playwright - cross-browser, codegen, trace

The Simple Answer

Playwright (Microsoft) is the newest and fastest-growing E2E testing framework. It supports all browsers from a single API: Chromium (Chrome, Edge), Firefox, and WebKit (Safari). This is its defining advantage - true cross-browser testing without rewriting tests.

Killer Features

Codegen - `npx playwright codegen https://your-app.com` opens a browser. Every click, type, and navigation you perform is recorded as test code. The fastest way to create tests - record the happy path, then add assertions and edge cases manually.

Trace viewer - When a test fails, Playwright saves a trace file containing: DOM snapshots at every step, network request/response pairs, console log entries, and screenshots. Open with `npx playwright show-trace trace.zip` - a complete visual replay of the test execution. This makes debugging CI failures possible without reproducing locally.

Auto-wait - Like Cypress. All actions automatically wait for: the element to be visible, enabled, stable (not animating), and able to receive events. No explicit waits needed.

API testing built-in - Test APIs without a browser: `const response = await request.get('/api/users');`
`expect(response.ok()).toBeTruthy();`. Faster than browser tests when you only need to verify API responses.

Q24 Cypress vs Selenium vs Playwright

The Comparison

Feature	Selenium	Cypress	Playwright
Browser support	All browsers	Chrome, Firefox	All browsers
Languages	Java, Python, JS, C#, Ruby	JavaScript/TypeScript only	JS, Python, Java, .NET

Architecture	External WebDriver protocol	Runs inside the browser	Browser DevTools Protocol
Auto-wait	No (manual waits required)	Yes (built-in)	Yes (built-in)
Speed	Slowest	Fast	Fastest
Debugging	Screenshots on failure	Time travel (DOM snapshots)	Trace viewer (full replay)
Network mocking	Not built-in	cy.intercept()	route.fulfill()
Parallel execution	Selenium Grid	Cypress Cloud (paid)	Built-in (free)
Mobile testing	Appium integration	No	Device emulation
Maturity	20+ years, most established	7+ years, stable	4+ years, growing rapidly

****The 2025+ recommendation:**** Playwright for new projects - best browser support, fastest execution, best debugging (trace viewer), free parallelization, and multi-language support. Cypress if your team is already invested and happy with it. Selenium only for: teams needing multi-language support that Playwright does not cover, mobile testing via Appium, or legacy integration.

Q25 Postman advanced - scripts, chaining

Pre-Request Scripts

Run JavaScript before sending the request. Generate dynamic data: `pm.variables.set("timestamp", Date.now());`. Calculate authentication: generate HMAC signatures, JWT tokens. Fetch prerequisites: `pm.sendRequest()` calls another endpoint to get an auth token, stores it for the main request.

Response Chaining

Capture data from one response and use in the next request. Test workflow: "Create User" → capture `user_id` from response → use `{{user_id}}` in "Get User" → use `{{user_id}}` in "Delete User." Implementation: `pm.environment.set("user_id", pm.response.json().id);` in the test script of "Create User." `{{user_id}}` is automatically resolved in subsequent requests.

Data-Driven Testing

Import a CSV or JSON file with test data. Newman runs the entire collection once PER ROW of data. Test the same API with 100 different inputs: `newman run collection.json -d test-data.csv --iteration-count 100`. Each iteration uses a different row of data. Results show which data rows caused failures.

Q26 API contract testing - Pact

The Simple Answer

Contract testing verifies that the API producer and consumer AGREE on the request/response format without running end-to-end tests. Each side tests independently against a shared contract.

The Analogy

In international trade, a contract specifies: what goods will be shipped, in what format, by what date. The buyer tests that their warehouse can receive the specified goods. The seller tests that they can produce the specified goods. Neither needs the other present for their test. If either side changes the contract without agreement, the test fails. Contract testing applies this same principle to APIs.

How Pact Works

Consumer side (e.g., frontend): Writes a test that mocks the provider and defines expected interactions: "When I GET /users/1, I expect: status 200, body with {id: number, name: string, email: string}." The test generates a Pact file - a JSON contract.

Provider side (e.g., backend): replays the Pact file against the real provider implementation. Verifies that every consumer expectation is met: "Can I return status 200 with {id, name, email} for GET /users/1? Yes - contract fulfilled."

Pact Broker: Central repository for contracts. Tracks which consumer versions are compatible with which provider versions. "Can I deploy provider v2.5?" - the broker checks all consumer contracts against v2.5 and returns yes/no.

Why Not Just E2E Tests

E2E tests are slow (minutes), flaky (network, timing, data), and expensive (need all services running). Contract tests are fast (milliseconds - they mock one side), reliable (no network), and independent (each side tests alone). A broken contract is caught in seconds during a PR pipeline, not after a 30-minute E2E suite on a fragile staging environment.

Q27 Performance testing - JMeter, k6, Locust

Test Types

Load test - Expected traffic volume. "Can the system handle 1000 concurrent users?" Verifies the system meets its SLAs under normal conditions. Stress test - Beyond expected. "At what point does the system break? 2000? 5000? 10000 users?" Identifies the breaking point and failure mode (graceful degradation vs crash). Soak test - Sustained load over hours or days. "Does the system degrade over time?" Catches: memory leaks, connection pool exhaustion, log file growth, database bloat. Spike test - Sudden traffic burst. 100 users → 5000 users → 100 users in 30 seconds. "Does the system handle sudden traffic and recover afterward?"

Tool Comparison

k6 (Grafana) - JavaScript-based. Developer-friendly. Script load tests like code. CLI output with Grafana dashboards. Excellent CI integration. The modern choice for DevOps teams.

JMeter - Java-based. GUI for test creation (drag-and-drop). Heavyweight but supports every protocol (HTTP, JDBC, LDAP, FTP, SOAP). The traditional enterprise choice. Can be complex for simple HTTP load tests.

Locust - Python-based. Define user behavior in Python code. Web UI for real-time monitoring. Good for Python teams who want test logic in familiar syntax.

Gatling - Scala-based. Code-driven with excellent HTML reports. High performance. Good for JVM teams.

****The recommendation:**** k6 for most DevOps teams. Write load tests in JavaScript, version-control alongside application code, run in CI, visualize with Grafana. k6 is becoming the standard.

Q28 Chaos testing - Chaos Monkey, Litmus, Game Days

The Simple Answer

Chaos testing intentionally injects failures to verify that systems recover gracefully. Instead of waiting for production to break (and discovering your monitoring, alerting, and runbooks are inadequate), you break things on purpose in a controlled way.

The Analogy

Fire drills do not prevent fires - they verify that people know what to do when a fire occurs. Chaos testing is a fire drill for your infrastructure. You do not wait for the database to crash at 3 AM - you crash it at 2 PM on a Tuesday when the team is ready, and verify that failover works, alerts fire, and the runbook is accurate.

Tools

Litmus (CNCF) - Kubernetes-native chaos engineering. ChaosExperiment CRDs define what to break: pod-kill, network-latency, disk-fill, node-drain. Integrates with CI/CD - run chaos experiments as pipeline stages. Chaos Monkey (Netflix) - Randomly terminates EC2 instances in production. Forces teams to build resilient systems - if your service cannot survive an instance dying, Chaos Monkey will find out. Gremlin - Commercial SaaS platform. Attack categories: resource (CPU, memory, disk), network (latency, packet loss, DNS), state (process kill, time travel). AWS Fault Injection Simulator - Managed chaos for AWS. Inject faults into EC2, ECS, EKS, RDS, and networking.

Game Days

Scheduled chaos exercises with the entire team. Plan: what will we break, what do we expect to happen. Execute: inject the failure. Observe: do alerts fire? Do runbooks work? Does auto-healing work? Does the system recover within the SLA? Retrospective: what worked, what did not, what do we fix. Run quarterly at minimum.

Q29 Mocking - WireMock, MockServer

The Simple Answer

Mocks simulate external dependencies so you can test YOUR service in isolation. Instead of calling the real payment API (which costs money, is slow, rate-limited, and may be down), mock it - define what requests to expect and what responses to return.

WireMock

An HTTP mock server. Start it: `java -jar wiremock-standalone.jar`. Define stubs: when GET /api/users/1, return { "name": "Alice", "email": "alice@example.com" } with status 200. Verify: was the expected request made? How many times? With what headers? Run as a standalone server, embedded in JUnit tests, or as a Docker container.

When to Mock vs When to Use Real Services

Mock: External third-party APIs (payment gateways, SMS providers). Services you do not control. Expensive-to-call APIs. Flaky or slow dependencies. Use real (Testcontainers): Databases (SQL behavior is hard to mock accurately). Message queues (delivery semantics matter). Caches (eviction behavior matters). The rule: Mock what you do not own. Test with real versions of what you do own.

Q30 Testcontainers - real services for integration

The Simple Answer

Testcontainers spins up real databases, message queues, and services in Docker containers for integration tests. Instead of mocking PostgreSQL, run a real PostgreSQL container. Tests interact with a real database - catching issues that mocks miss.

The Analogy

Mocking a database is like practicing driving in a video game. It teaches the basics but misses real-world physics. Testcontainers is like practicing on a real track - the car behaves exactly as it will in production. SQL syntax errors, transaction deadlocks, constraint violations, and encoding issues are caught during testing, not in production.

How It Works

Java: `@Container static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>("postgres:15")`. JUnit starts the container before tests, provides the JDBC URL, and destroys the container after tests. Python: with `PostgresContainer("postgres:15")` as `postgres`: `conn = psycopg2.connect(postgres.get_connection_url())`.

Supported Services

PostgreSQL, MySQL, MariaDB, MongoDB, Redis, Kafka, RabbitMQ, Elasticsearch, MinIO (S3-compatible), LocalStack (all AWS services), Keycloak, Vault, and 50+ more. Any Docker image can be wrapped as a Testcontainer.

****Testcontainers changed integration testing.**** Before: shared test databases with stale data, conflicting tests, "works on my machine" issues. After: each test gets a fresh, isolated database in seconds. No shared state. No cleanup scripts. No flaky tests from data contamination. This is the modern standard for integration testing.

Q31 Test reporting - Allure

The Simple Answer

Test reports transform raw test output into visual dashboards for humans and CI tools.

Allure - Rich interactive HTML reports with: test categorization by suite/severity/feature, step-by-step execution with attachments (screenshots, logs), history trends (pass rate over time), environment info (OS, browser, version), and failure analysis (which tests failed and why). Integrates with: JUnit, PyTest, Mocha, Cypress, Playwright, TestNG. The standard for comprehensive test reporting.

JUnit XML - Machine-readable format consumed by every CI tool. Jenkins, GitLab CI, GitHub Actions, and Azure DevOps all parse JUnit XML natively to show test results in the pipeline UI. Most test frameworks export JUnit XML: `pytest --junitxml=results.xml`, `mocha --reporter mocha-junit-reporter`.

Coverage reports - HTML reports showing which lines are covered (green) and uncovered (red). Tools: JaCoCo (Java), coverage.py (Python), Istanbul/c8 (JavaScript). Upload to SonarQube for quality gate evaluation or to Codecov/Coveralls for PR decoration.

Q32 Flaky tests - causes, detection, quarantine

The Simple Answer

A flaky test passes sometimes and fails sometimes with no code change. Flaky tests are the #1 destroyer of CI trust. When developers cannot trust the test suite, they start ignoring failures - "oh, that test is always flaky." This is how real bugs slip through.

The Analogy

A flaky smoke detector that goes off randomly trains you to ignore it. When a real fire starts, you dismiss the alarm. Flaky tests train developers to ignore CI failures. When a real bug fails the suite, it is dismissed as "probably that flaky test again."

Common Causes and Fixes

Timing/race conditions - Test assumes an element appears within 2 seconds, but under CI load it takes 3. Fix: use explicit waits with sufficient timeout, never `time.sleep()`. Shared mutable state - Test A creates data that Test B accidentally depends on. When test order changes, B fails. Fix: each test creates its own data and cleans up. Use Testcontainers for database isolation. External dependency - Test calls a real external API that is occasionally slow or returns different data. Fix: mock external calls with WireMock. Order dependency - Test only passes when run after a specific other test. Fix: `pytest --randomly` detects this. Each test must be independent.

Management Strategy

Detect: Track pass rates per test. A test with less than 99% pass rate is flaky. Tools: Allure TestOps, BuildPulse, Launchable, or custom tracking. Quarantine: Move confirmed flaky tests to a separate suite that does NOT block CI. Mark with `@pytest.mark.flaky` or `@Disabled("Flaky - ticket JIRA-456")`. Fix: Root cause analysis. Apply the appropriate fix from the causes above. Verify with 100+ consecutive runs - zero failures means resolved. SLA: No flaky test stays quarantined for more than 7 days. Either fix it or delete it. Quarantine is treatment, not a permanent solution.

Q33 Test data management

The Simple Answer

Test data quality directly determines test quality. Bad test data causes: false positives (tests pass with wrong data), false negatives (tests fail because data is stale), and flaky tests (data from previous runs interferes).

Strategies

Factories - Generate test data programmatically. Factory Boy (Python): `user = UserFactory(name="Alice", role="admin")`. Faker generates realistic data: `fake.email()`, `fake.address()`, `fake.phone_number()`. Each test gets unique data - no conflicts with other tests.

Fixtures - Predefined JSON or SQL files loaded before tests. Predictable and reproducible. Limitation: rigid - adding a database column requires updating all fixtures.

Data masking - Copy production data with PII replaced by realistic fakes. Preserves: data volume, distribution patterns, foreign key relationships, and edge cases. Tools: Tonic, Delphix, pg_anonymize. Used when tests need production-scale data to catch performance issues or data-dependent bugs.

Database per test - Testcontainers creates a fresh database for each test. Complete isolation. No cleanup needed. No data contamination. The gold standard for integration test data management.

Q34 Security testing in pipeline

The Simple Answer

Security testing integrates into the quality gate alongside functional tests. Each pipeline stage adds a security check.

Pipeline Integration Points

Pre-commit: gitleaks and git-secrets scan for accidentally committed credentials (API keys, passwords, private keys). Prevents secrets from entering the repository.

Build stage (SAST): Static Application Security Testing. Semgrep, CodeQL, or SonarQube scan source code for vulnerabilities (SQL injection patterns, XSS, insecure deserialization). Fast - analyzes code without running it. Catches: hardcoded secrets, unsafe function calls, injection patterns.

Build stage (SCA): Software Composition Analysis. Snyk, Dependabot, or Trivy scan dependencies for known CVEs. "This project uses log4j 2.14.1 which has CVE-2021-44228 (Critical)." Catches: vulnerable dependencies that you did not write but your application includes.

Post-build: Container image scanning. Trivy, Snyk Container, or AWS ECR scanning analyze the built Docker image for: OS package vulnerabilities, application dependency vulnerabilities, and configuration issues.

Staging (DAST): Dynamic Application Security Testing. OWASP ZAP tests the running application by sending malicious requests. Catches: vulnerabilities that only appear at runtime (misconfigurations, exposed endpoints, missing security headers).

****The quality gate integration:**** Each security tool produces findings with severity levels. The quality gate defines thresholds: "0 Critical, 0 High on new code. Medium and below can be tracked." Pipeline fails if thresholds are exceeded - just like test failures block merging, security findings above the threshold block deployment.

Q35 Infrastructure testing - Molecule, InSpec, Goss, Terratest

The Simple Answer

Infrastructure testing verifies that your infrastructure automation (Ansible roles, Terraform modules, Dockerfiles) produces the correct result.

Molecule - Tests Ansible roles. Workflow: create a test environment (Docker container or VM) → apply the Ansible role (converge) → verify the result (Testinfra or Goss) → destroy. Run in CI on every PR that changes an Ansible role. Catches: broken playbooks, missing handlers, incorrect template rendering.

InSpec - Infrastructure compliance testing. Ruby DSL. Tests: describe package('nginx') do; it { should be_installed }; end. describe port(80) do; it { should be_listening }; end. CIS benchmark profiles test compliance with security standards. Produces audit-ready reports.

Goss - YAML-based server testing. Fast Go binary. Tests: packages installed, services running, files existing with correct permissions, ports listening, command outputs matching expected values. Integrates with Molecule as a verifier. Simpler than InSpec for basic checks.

Terratest - Tests Terraform code by DEPLOYING REAL infrastructure, running assertions against it, then destroying it. Written in Go. The most realistic Terraform testing - verifies actual cloud resources, not just plan output. Trade-off: slow (creates real resources), costs money (cloud charges), but catches issues that plan-based testing misses (IAM permission errors, networking issues, service limits).

SECTION 3 - Senior Architecture (Q36–Q42)

Q36 Testing strategy - what to test at which level

The Decision Framework

Unit test: Pure business logic - calculations, transformations, validations, parsing, formatting. Fast, many, specific. Target: 80%+ coverage on business logic.

Integration test (Testcontainers): Component interactions - database queries, API client calls, message publishing, cache operations. Test YOUR code with REAL dependencies. Not with other services.

Contract test (Pact): API interfaces BETWEEN services. Each service tests its contracts independently. Fast (no real service needed). Prevents interface breaks.

E2E test (Playwright): ONLY critical user flows - the flows that generate revenue or are regulatory requirements. Login → checkout → payment. Account creation → verification. 5-10 critical flows maximum. Maintain ruthlessly.

Performance test (k6): Key endpoints under load. Run nightly or before releases. Establish baselines. Alert on regression.

Chaos test (Litmus): Failure scenarios. Monthly Game Days or continuous chaos in staging. Kill pods, inject latency, fill disks.

Risk-Based Testing

Not every component deserves equal testing effort. Payment processing: 90%+ coverage, comprehensive contract tests, dedicated performance tests, chaos testing. Internal admin dashboard: 60% coverage, basic smoke tests. One-time migration scripts: manual testing may suffice. Allocate testing effort proportional to business risk and change frequency.

Q37 Quality gates design - balancing speed and quality

The Four Gates

Gate 1 - Pre-merge (PR pipeline, target under 10 minutes): Unit tests, linting (ESLint/Ruff), SAST (Semgrep), SCA (Snyk), SonarQube analysis on new code, contract tests. Fast. Runs on every commit. Blocks merge on failure. This gate catches 90% of issues.

Gate 2 - Post-merge (main pipeline, target under 30 minutes): Full integration tests (Testcontainers), image build, container scan (Trivy), contract verification against all consumers. Runs on merge to main. Blocks deployment on failure.

Gate 3 - Pre-production (target under 1 hour): DAST against staging (OWASP ZAP), performance tests on key scenarios (k6), infrastructure compliance scan (InSpec). Runs before production deployment. Blocks production release on failure.

Gate 4 - Post-production (continuous): Canary analysis (error rate, latency from Prometheus), synthetic monitoring (Playwright tests running against production every 5 minutes), smoke tests after deployment. Automatic rollback on failure.

****The balance principle:**** Too many gates = slow pipelines = developer frustration = people gaming the system (disabling tests, merging without review). Too few gates = bugs in production = incidents = customer impact. The

goal: catch 95% of issues in Gate 1 (fast, cheap) and only 5% in Gates 2-4 (slower, more expensive). If Gate 1 takes more than 10 minutes, developers will stop running it.

Q38 Test automation ROI

What to Automate

Tests that run frequently (every PR, every deploy). Tests that are deterministic (same input always produces same output). Tests that take significant manual effort (regression suites with hundreds of cases). Tests for critical paths (payment, authentication, data integrity).

What to Keep Manual

Exploratory testing (discovering unexpected behavior through creative investigation). Usability testing (human judgment about user experience). One-time verifications (database migration validation). Tests in rapidly changing areas (UI under active redesign - the automation breaks faster than it can be maintained).

ROI Calculation

A manual regression test taking 4 hours, run twice per week, at \$50/hour costs \$20,800/year. Automating it takes 40 hours to build (one-time \$2,000) and 4 hours/month to maintain (\$2,400/year). Year 1 ROI: $\$20,800 - \$2,000 - \$2,400 = \$16,400$ saved. The automation pays for itself in less than 2 months.

Q39 Testing microservices

The Challenge

50 microservices with complex interaction patterns. Running all services for E2E tests is slow, flaky, and expensive. The testing strategy must adapt to distributed architecture.

The Microservice Testing Approach

Per-service (in the service's own pipeline): Unit tests - high coverage, fast. Integration tests - Testcontainers for the service's own database, cache, and queue. Contract tests - Pact consumer tests defining what this service expects from others.

Cross-service (in a shared pipeline): Pact provider verification - each provider verifies all consumer contracts before deploying. Pact Broker tracks compatibility - "Can I deploy service A v2.5 given that service B is at v1.3?" E2E tests - 5 critical flows through the full system. Run nightly against a full staging environment. Accept some flakiness. Keep this suite SMALL.

Service virtualization: Mock dependent services for testing. WireMock stubs replicate other services' APIs. Test one service without deploying the other 49. Record real traffic and replay it as stubs - realistic mock behavior.

Q40 Testing in production

The Simple Answer

Some things can only be tested in production - real traffic patterns, real data volumes, real user behavior, real network conditions, and real infrastructure quirks. Testing in production is not reckless - it is pragmatic when done with proper safeguards.

Techniques

Canary deployment - Deploy new version to 5% of traffic. Monitor error rates and latency via Prometheus/Datadog. If healthy after 10 minutes, expand to 25%, 50%, 100%. If unhealthy, automatic rollback. Testing with real users, limited blast radius. See K8s Ecosystem guide Q42 (Argo Rollouts).

Feature flags - Deploy code with the feature disabled. Enable for internal users first. Then 1% of real users. Then 10%. Monitor each expansion. Decouple deployment (code reaches production) from release (users see the feature).

Synthetic monitoring - Automated Playwright tests running in production every 5 minutes from multiple geographic locations. Test critical flows: login, search, checkout. Catches: CDN issues, DNS problems, regional outages. Alerts when a synthetic test fails.

Observability-driven testing - Deploy and watch metrics. If error rate increases, if latency degrades, if business metrics drop (conversion rate, revenue) - the deployment is bad. The metrics ARE the test. This requires mature observability (see Monitoring guide).

Q41 Compliance testing

The Simple Answer

Regulated industries (finance, healthcare, government) require evidence that software meets specific standards. Compliance testing automates the production of this evidence, replacing manual audit preparation with continuous automated verification.

Implementation

Automated compliance scans - AWS Config rules verify cloud resources match compliance requirements. Azure Policy enforces organizational standards. OpenSCAP scans servers against CIS benchmarks. Run continuously - not once before an audit.

Test traceability - Link each regulatory requirement to automated test(s). Requirement REQ-AUTH-001 ("Users must authenticate with MFA") links to: unit tests for MFA code generation, integration tests for MFA verification flow, E2E test for the full MFA login process. Auditors see: requirement → test → passing result → evidence timestamp.

Immutable audit reports - Allure, SonarQube, and compliance tools produce HTML/PDF reports. Store reports in S3 with object lock - immutable, tamper-proof. When auditors arrive, the evidence is already collected and organized.

Q42 Testing culture

The Simple Answer

Testing is not a phase, a team, or an afterthought - it is a practice embedded in every part of software delivery. The most effective testing cultures share these traits.

Cultural Principles

Developers own quality. The person who wrote the code writes the tests. Not a separate QA team. Developers understand their code best and write the most effective tests. QA focuses on strategy, tooling, exploratory testing, and test automation infrastructure - not manual test execution.

Definition of Done includes tests. A feature is not "done" until it has: passing unit tests, integration tests for component interactions, updated contract tests if APIs changed, and documentation. No exceptions.

Test failures are sacred. A failing test blocks deployment. Period. No "we'll fix it in the next sprint." No "it's probably flaky." If it is flaky, fix or quarantine it. If it is a real failure, fix the code. Trust in the test suite is non-negotiable.

Flaky tests are P1 bugs. They erode CI trust and must be fixed within 7 days. A flaky test that stays flaky teaches the team to ignore failures.

Testing metrics matter. Track: coverage (is it improving?), flaky test count (is it decreasing?), time-to-green (is the pipeline fast?), test execution time (is it under 10 minutes?), and defect escape rate (how many bugs reach production?).

SECTION 4 - Tricky Scenarios (Q43–Q50)

Q43 SonarQube gate blocking release

Step-by-Step

Step 1 - Understand what failed. Open the SonarQube dashboard. Which condition failed? Coverage below 80%? New bugs? New vulnerabilities? Duplication above 3%?

Step 2 - If coverage failed. The new code lacks tests. Identify the uncovered lines (SonarQube highlights them). Write tests targeting the business logic - meaningful assertions, not just calling functions to increase the percentage. Focus on branches (if/else paths) not just lines.

Step 3 - If bugs or vulnerabilities were found. These are real issues SonarQube detected through static analysis. A null dereference, a resource leak, an SQL injection pattern. Fix them - they would become production incidents or security breaches.

Step 4 - If the issue is a false positive. SonarQube is not perfect - some findings are false positives. Mark as "Won't Fix" or "False Positive" in SonarQube with a written justification. The gate re-evaluates. Do not abuse this - if you have many false positives, create exclusion rules or customize the quality profile.

Step 5 - Emergency release under extreme time pressure. A critical hotfix needs to ship NOW. The quality gate fails on a minor issue unrelated to the hotfix. Bypass with: management approval, documented justification, and a follow-up ticket to fix the issue within 48 hours. NEVER make bypassing the standard - it should be rare, documented, and uncomfortable.

Q44 Flaky E2E test failing randomly

Step-by-Step

Step 1 - Quarantine immediately. Move the test to a non-blocking suite so it does not block the entire team. Create a ticket: "FLAKY: checkout_flow_test - quarantined [date], must fix by [date + 7 days]."

Step 2 - Reproduce. Run the test 50-100 times in a loop: `for i in $(seq 1 100); do npx playwright test checkout.spec.ts; done`. Record the failure rate and the error messages. Does it fail at the same step every time? Under load only? After a specific test?

Step 3 - Diagnose. Common patterns: Element not found (race condition - page not fully loaded) → check auto-wait configuration, add explicit wait for specific element. Timeout exceeded (API slow under CI load) → mock the API response or increase timeout. Different data than expected (shared test data modified by another test) → isolate test data with factories or Testcontainers. Screenshot on failure shows unexpected modal/popup → handle the modal in the test setup.

Step 4 - Fix and verify. Apply the fix. Run 100 times consecutively. Zero failures = resolved. Return to the main suite. If still flaky after analysis, consider: is this E2E test testing something better covered by a unit or integration test? If yes, replace it.

Q45 Coverage dropped from 80% to 45%

Step-by-Step

Step 1 - Identify the cause. git diff between the last passing build and the current one. Likely: a large new feature with zero tests, or a merge of generated code (protobuf stubs, API clients, database migrations).

Step 2 - If new feature code. The developer skipped tests. Write tests - focus on business logic first (service layer, domain logic, utilities). Do not waste time testing framework boilerplate (controllers that just delegate to services, entity getters/setters).

Step 3 - If generated code. Generated code should not count toward coverage metrics. Add exclusions: in sonar-project.properties: sonar.coverage.exclusions=/generated/, /proto/, /migrations/. In .coveragerc (Python): omit = */generated/*, */migrations/*.

Step 4 - Prevent recurrence. Set SonarQube quality gate to enforce coverage on NEW CODE (not overall). Default Sonar Way gate requires 80% on new code. Even if the overall coverage is 45%, every new change must meet 80%. Over time, coverage rises naturally as new tested code replaces old untested code.

Q46 Selenium tests taking 2 hours

Step-by-Step

Step 1 - Parallelize. Run tests across multiple browser instances simultaneously. Selenium Grid distributes tests to a pool of browsers. CI: 4 parallel jobs, each running 25% of the suite. 2 hours becomes 30 minutes.

Step 2 - Apply the testing pyramid. Many E2E tests are likely testing logic better covered by unit or integration tests. Example: an E2E test that fills a form with 20 different invalid inputs to test validation - this should be 20 unit tests (milliseconds each) and 1 E2E test (the happy path). Audit each E2E test: does this NEED a real browser? If not, move it down the pyramid.

Step 3 - Speed up individual tests. Replace time.sleep() with explicit waits (each sleep(3) wastes up to 3 seconds). Use headless browser mode (no GPU rendering - 30% faster). Optimize test data setup: create test data via API calls (milliseconds) instead of navigating the UI (seconds). Pre-authenticate via API cookies instead of going through the login form for every test.

Step 4 - Consider migration. Playwright is 2-3x faster than Selenium for the same tests. The auto-wait eliminates most timing-related slowness. Migration effort varies - evaluate with a pilot of 10 tests and measure the time difference.

Q47 API contract broken after deployment

What Happened

Service A (consumer) calls Service B (producer) at GET /api/products/:id. Service B's team changed the response - renamed the "price" field to "amount" and removed the "category" field. Service B's own tests passed (they test their endpoints). Service A broke in production because it expected "price" and "category" to exist.

Step-by-Step

Step 1 - Rollback Service B to the previous version that returns "price" and "category." Production is restored.

Step 2 - Implement contract testing with Pact. Service A (consumer) writes a test: "I expect GET /api/products/1 to return a body with {price: number, category: string}." This generates a Pact contract file. Service B (provider) runs the Pact file against its implementation before deploying. The test would have caught the breaking change - "consumer expects 'price' but provider returns 'amount'."

Step 3 - Deploy the change properly. Service B adds "amount" alongside "price" (backward compatible). Service A updates to use "amount." Once all consumers have migrated, Service B removes "price." This is the expand-contract migration pattern.

Prevention

Every API change is verified against all consumer contracts in the PR pipeline. Breaking changes follow the expand-contract pattern: add new → migrate consumers → remove old. Pact Broker tracks compatibility between service versions - "can I deploy?" is answered automatically.

Q48 Performance test shows 10x latency under load

Step-by-Step

Step 1 - Reproduce and profile. Run k6 with 100 concurrent users while monitoring CPU, memory, disk I/O, network, and database metrics. Identify which resource hits its limit first - that is your bottleneck.

Step 2 - Database bottleneck (most common). Slow queries under load: run EXPLAIN ANALYZE on the slowest queries. Missing indexes? N+1 query pattern (1 query for the list + 1 query per item)? Connection pool exhausted (max_connections reached, new queries queue)? Fix: add indexes, batch queries, increase pool size, add read replicas for read-heavy workloads.

Step 3 - Application bottleneck. CPU at 100%: profile with async-profiler (Java), py-spy (Python), or clinic.js (Node). Find the hot function - it might be: inefficient serialization, regex backtracking, or unnecessary computation. Memory growing linearly: memory leak - objects allocated but never garbage collected. Common in long-running processes.

Step 4 - Infrastructure bottleneck. Instance too small: vertical scale (larger instance) or horizontal scale (more replicas behind load balancer). Network: cross-AZ latency, bandwidth saturation. Disk: IOPS limit reached for database writes.

Step 5 - Document. Record the performance baseline: p50=20ms, p95=50ms, p99=100ms at 500 concurrent users. Maximum capacity: 2000 concurrent users before degradation. Share with the team. Set performance SLOs. Run performance tests nightly to catch regressions early.

Q49 Test environment data corrupted

What Happened

Tests pass with wrong data - the test asserts "user count is 10" but it should be 5. The test passes with the corrupted data and misses a real bug. Or: a test that was passing starts failing because another team's test modified shared data.

Step-by-Step

Step 1 - Restore to known state. Reload seed data from version-controlled fixtures. Or destroy and recreate the test database from scratch (docker compose down && docker compose up).

Step 2 - Root cause. Who or what corrupted the data? A developer manually modified the test database? A test that does not clean up after itself? A shared database used by multiple teams' CI pipelines simultaneously?

Step 3 - Permanent fixes. Use Testcontainers - each test run gets a fresh, isolated database in a Docker container. No shared state possible. Use database transactions - start a transaction before each test, rollback after. The database returns to its pre-test state automatically. Seed data is in version control and loaded programmatically - never manually modified. Separate databases per team - do not share test databases across CI pipelines.

Q50 Design testing strategy for 30 microservices

The Complete Architecture

Per-service testing (in each service's pipeline):

Unit tests - 80%+ coverage on business logic. Run in PR pipeline. Target: under 2 minutes. Integration tests - Testcontainers with the service's own database and cache. Run in PR pipeline. Target: under 5 minutes. API contract tests - Pact consumer tests for each dependency. Run in PR pipeline. Target: under 1 minute. SonarQube quality gate on new code - 0 bugs, 0 vulnerabilities, over 80% coverage, under 3% duplication.

Cross-service testing:

Pact Broker - Central contract repository. Provider verification runs in each provider's pipeline. "Can I deploy?" check before any deployment. E2E tests - 5 critical user flows (login, checkout, payment, account creation, search) run nightly against a full staging environment. Target: 15 minutes maximum. Accept minor flakiness - investigate and fix promptly. Performance tests - k6 tests on the 10 highest-traffic endpoints. Run nightly. Establish baselines. Alert on regression.

Infrastructure testing:

Terraform: terraform validate + tfsec in every PR. Terratest for critical modules (VPC, EKS, RDS) nightly. Ansible: Molecule tests for every role. Run in PR pipeline. Docker: hadolint for Dockerfile linting. Trivy for image scanning.

Production testing:

Canary deployments with Argo Rollouts and Prometheus-based analysis. Automatic rollback when error rate exceeds 1% or p99 latency exceeds 500ms. Synthetic monitoring - 5 critical flows tested every 5 minutes from 3 geographic regions. Chaos testing - monthly Game Days with Litmus. Kill random pods, inject network latency, fill disks.

Quality gates summary:

Gate 1 (PR, under 10 min): unit + integration + contract + lint + SAST + SCA. Gate 2 (main, under 30 min): full tests + image build + image scan + DAST. Gate 3 (deploy, continuous): canary analysis + synthetic tests. Gate 4 (ongoing): monthly chaos testing + quarterly security review.

Metrics to track:

Coverage: greater than 80% on new code across all services. Flaky tests: 0 (7-day quarantine SLA). PR pipeline: under 10 minutes. Contract test pass rate: 100%. Defect escape rate: under 2% (less than 2% of bugs reach production). MTTR for test failures: under 4 hours.

Interview Tips

For Junior Roles

Focus on Q1-Q15. Know the testing pyramid and explain why more unit tests. Describe TDD. Write basic JUnit or PyTest tests. Know SonarQube quality gates. Explain why E2E tests are expensive and flaky.

For Mid-Level Roles

Q1-Q35. Be ready to discuss Cypress vs Playwright vs Selenium (and recommend Playwright for new projects). Explain contract testing with Pact. Know performance testing types and when to use k6. Describe Testcontainers and why they replaced mocked databases. Explain how to handle flaky tests.

For Senior Roles

All 50 questions. Testing strategy design, microservices testing patterns, testing in production, quality gate architecture, and building a testing culture separate senior from mid-level. Be ready to design a complete testing strategy for a complex platform (Q50).

Quick Reference Cheat Sheet

Topic	Key Takeaway
Testing pyramid	Many unit (70%), fewer integration (20%), fewest E2E (10%).
TDD	Red (fail) then Green (pass) then Refactor. Test BEFORE code.
BDD	Given/When/Then. Business language. Cucumber/Behave. Living documentation.
Coverage	80% target. Branch is better than line. High coverage is not good tests.
SonarQube	Quality gates on new code. 0 bugs, 0 vulns, over 80% coverage. PR decoration.
JUnit/PyTest	Annotations/fixtures. Mockito/pytest-mock for isolation. Parametrize for multiple inputs.
Selenium	Oldest E2E. All browsers. Manual waits. Page Object Model. Slowest.
Cypress	In-browser. Auto-wait. Time travel. Network intercepts. JS only.
Playwright	All browsers. Codegen. Trace viewer. Fastest. Best for new projects 2025+.
Postman/Newman	API testing. Collections + environments. Newman for CI automation.
Contract testing	Pact. Consumer defines, producer verifies. Fast, prevents API breaks.
Performance	k6 for modern teams. Load/stress/soak/spike types. Nightly runs.
Chaos testing	Litmus for K8s. Game Days quarterly. Verify resilience before production breaks.
Testcontainers	Real databases in Docker for tests. No mocks. No shared state. Gold standard.
Flaky tests	Quarantine immediately. Fix within 7 days or delete. 100 runs to verify.
Quality gates	PR under 10 min. Main under 30 min. Staging under 1 hour. Production continuous.
Microservices	Unit + contract per service. E2E only for critical flows. Pact Broker for compatibility.
Testing in prod	Canary + feature flags + synthetic monitoring + observability. Not reckless, pragmatic.
Infrastructure	Molecule (Ansible), Terratest (Terraform), InSpec/Goss (compliance).
Culture	Devs own quality. Test failures block deploy. Flaky tests are P1. No exceptions.